

# Phase 1 Test Specifications

## POC-01 — Loan Application Management System

**Total Test Cases:** 20 | **Pass Threshold:** 14 of 20 (70%)

### Test Case Format

Each test case follows: Prerequisites → Test Data → Steps → Expected Result → Pass/Fail Criteria → Code Skeleton

### UNIT TESTS (8 cases)

TC-01-P1-UNIT-01: Create Applicant with Valid Data

**Category:** Unit | **Priority:** High **Prerequisites:** Database initialized, application under test imported

**Test Data:**

```
valid_applicant = {
  "name": "Priya Sharma",
  "email": "priya@example.com",
  "phone": "9876543210",
  "credit_score": 720,
  "annual_income": 600000.0,
  "employment_status": "salaried"
}
```

**Steps:**

- 1. Call `applicant_service.create_applicant(db, valid_applicant)`
- 2. Retrieve the created applicant from the database by ID

**Expected Result:** Applicant created with correct field values, ID assigned, created\_at populated

**Pass Criteria:** Returned object has all fields matching input; `id` is not None; `created_at` is not None

**Fail Criteria:** Exception raised; ID is None; any field value differs from input

```
def test_create_applicant_valid(db_session):
    # Arrange
    data = CreateApplicantSchema(
        name="Priya Sharma", email="priya@example.com",
        phone="9876543210", credit_score=720,
        annual_income=600000.0, employment_status="salaried"
    )
```

```
# Act
applicant = applicant_service.create_applicant(db_session, data)
# Assert
assert applicant.id is not None
assert applicant.email == "priya@example.com"
assert applicant.credit_score == 720
assert applicant.created_at is not None
```

---

## TC-01-P1-UNIT-02: Reject Invalid Email Format

**Category:** Unit | **Priority:** High **Prerequisites:** Pydantic schema imported

**Test Data:** `email = "not-an-email"`

**Steps:**

1. Attempt to create a `CreateApplicantSchema` with `email="not-an-email"`

**Expected Result:** `ValidationError` raised by Pydantic

**Pass Criteria:** `pydantic.ValidationError` is raised with field error for `email`

**Fail Criteria:** No exception raised; wrong exception type raised

```
def test_create_applicant_invalid_email():
    with pytest.raises(ValidationError) as exc_info:
        CreateApplicantSchema(
            name="Test", email="not-an-email",
            phone="9876543210", annual_income=500000,
            employment_status="salaried"
        )
    assert "email" in str(exc_info.value).lower()
```

---

## TC-01-P1-UNIT-03: Calculate EMI Correctly

**Category:** Unit | **Priority:** High **Prerequisites:** EMI utility function implemented

**Test Data:** `principal=500000, annual_rate=12, tenure_months=36` **Expected EMI:** ₹16,607 ( $\pm$  ₹10 tolerance for floating point)

**Steps:**

1. Call `calculate_emi(principal=500000, annual_rate=12, tenure_months=36)`

**Expected Result:** Returns value between 16597 and 16617

**Pass Criteria:** Return value within  $\pm 10$  of 16607

**Fail Criteria:** Return value outside tolerance; function raises exception

```
def test_calculate_emi_correctly():
    from app.utils.finance import calculate_emi
    result = calculate_emi(principal=500000, annual_rate=12, tenure_months=36)
    assert 16597 <= result <= 16617, f"Expected ~16607, got {result}"
```

---

## TC-01-P1-UNIT-04: Validate Loan Amount Bounds

**Category:** Unit | **Priority:** High **Prerequisites:** Pydantic schema with amount validation

**Test Data (invalid):** `amount_requested=5000` (below minimum 10000), `amount_requested=15000000` (above maximum 10000000)

### Steps:

1. Attempt to create `CreateApplicationSchema` with `amount=5000`
2. Attempt to create `CreateApplicationSchema` with `amount=15000000`

**Expected Result:** `ValidationError` raised for both cases

**Pass Criteria:** `ValidationError` raised with field error for `amount_requested` in both cases

```
@pytest.mark.parametrize("amount", [5000, 15000000, -100, 0])
def test_validate_loan_amount_bounds(amount):
    with pytest.raises(ValidationError):
        CreateApplicationSchema(
            applicant_id=1, loan_type="personal",
            amount_requested=amount, tenure_months=24,
            purpose="Test"
        )
```

---

## TC-01-P1-UNIT-05: Accept Valid Status Transition

**Category:** Unit | **Priority:** High **Prerequisites:** Status transition validation logic implemented

**Test Data:** `current_status="submitted"`, `new_status="under_review"`

### Steps:

1. Call `validate_status_transition(current="submitted", new="under_review")`

**Expected Result:** Returns `True` or does not raise an exception

**Pass Criteria:** Function accepts the valid transition without error

```
def test_status_transition_valid():
    from app.services.application_service import validate_status_transition
    from app.models.application import ApplicationStatus
```

```
# Should not raise
assert validate_status_transition(
    ApplicationStatus.submitted,
    ApplicationStatus.under_review
) == True
```

---

## TC-01-P1-UNIT-06: Reject Invalid Status Transition

**Category:** Unit | **Priority:** High **Prerequisites:** Status transition validation logic implemented

**Test Data:** `current_status="approved", new_status="submitted"` (backward transition)

**Steps:**

1. Call `validate_status_transition(current="approved", new="submitted")`

**Expected Result:** Returns `False` or raises `InvalidTransitionError`

**Pass Criteria:** Function indicates the transition is invalid

```
def test_status_transition_invalid():
    from app.services.application_service import validate_status_transition
    from app.models.application import ApplicationStatus
    result = validate_status_transition(
        ApplicationStatus.approved,
        ApplicationStatus.submitted
    )
    assert result == False
```

---

## TC-01-P1-UNIT-07: Validate Document Type Enum

**Category:** Unit | **Priority:** Medium **Prerequisites:** DocumentType enum and schema defined

**Test Data (invalid):** `doc_type="passport_copy"` (not in valid enum)

**Steps:**

1. Attempt to create `CreateDocumentSchema` with `doc_type="passport_copy"`

**Expected Result:** `ValidationError` raised

**Pass Criteria:** `ValidationError` raised with error referencing `doc_type`

```
def test_document_type_validation_invalid():
    with pytest.raises(ValidationError):
        CreateDocumentSchema(
            application_id=1,
            doc_type="passport_copy",
```

```
        file_name="passport.pdf"
    )

def test_document_type_validation_valid():
    # Should not raise
    doc = CreateDocumentSchema(
        application_id=1, doc_type="id_proof", file_name="aadhaar.pdf"
    )
    assert doc.doc_type == "id_proof"
```

---

## TC-01-P1-UNIT-08: Credit Score Range Validation

**Category:** Unit | **Priority:** Medium **Prerequisites:** Applicant schema with credit\_score validation

**Test Data:** `credit_score=1200` (above max 900), `credit_score=100` (below min 300)

### Steps:

1. Attempt to create `CreateApplicantSchema` with `credit_score=1200`
2. Attempt to create `CreateApplicantSchema` with `credit_score=100`

**Expected Result:** `ValidationError` raised for both values

```
@pytest.mark.parametrize("score", [1200, 100, -1, 0])
def test_credit_score_validation_invalid(score):
    with pytest.raises(ValidationError):
        CreateApplicantSchema(
            name="Test", email="test@test.com", phone="9876543210",
            credit_score=score, annual_income=500000, employment_status="salaried"
        )
```

---

## API INTEGRATION TESTS (8 cases)

### Setup (conftest.py):

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.main import app
from app.database import Base, get_db

TEST_DATABASE_URL = "sqlite:///./test.db"

@pytest.fixture(scope="function")
def client():
    engine = create_engine(TEST_DATABASE_URL, connect_args={"check_same_thread":
```

```
False}})
    TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
    Base.metadata.create_all(bind=engine)

    def override_get_db():
        try:
            db = TestingSessionLocal()
            yield db
        finally:
            db.close()

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c

    Base.metadata.drop_all(bind=engine)

@pytest.fixture
def auth_token(client):
    # Register and login to get token
    client.post("/api/v1/auth/register", json={
        "name": "Test Officer", "email": "officer@test.com", "password":
"Test@1234"
    })
    response = client.post("/api/v1/auth/login", json={
        "email": "officer@test.com", "password": "Test@1234"
    })
    return response.json()["access_token"]

@pytest.fixture
def test_applicant(client, auth_token):
    response = client.post("/api/v1/applicants", json={
        "name": "Test Applicant", "email": "applicant@test.com",
        "phone": "9876543210", "credit_score": 720,
        "annual_income": 600000.0, "employment_status": "salaried"
    }, headers={"Authorization": f"Bearer {auth_token}"})
    return response.json()
```

## TC-01-P1-API-01: POST /applications Returns 201

**Category:** API | **Priority:** High **Prerequisites:** Running test client, valid auth token, applicant created

**Test Data:** Valid application payload

### Steps:

1. POST to `/api/v1/applications` with valid payload and auth header
2. Verify response status code
3. Verify response body structure

**Expected Result:** HTTP 201, response body contains `id`, `status="submitted"`, all input fields

**Pass Criteria:** Status code 201; `id` in response; `status == "submitted"`

```
def test_post_application_success(client, auth_token, test_applicant):
    response = client.post("/api/v1/applications", json={
        "applicant_id": test_applicant["id"],
        "loan_type": "personal",
        "amount_requested": 100000.0,
        "tenure_months": 24,
        "purpose": "Home renovation"
    }, headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 201
    data = response.json()
    assert data["id"] is not None
    assert data["status"] == "submitted"
    assert data["loan_type"] == "personal"
    assert data["amount_requested"] == 100000.0
```

---

TC-01-P1-API-02: POST /applications with Missing Fields Returns 422

**Category:** API | **Priority:** High **Prerequisites:** Running test client, valid auth token

**Test Data:** Payload missing required `loan_type` field

**Steps:**

1. POST to `/api/v1/applications` without `loan_type`

**Expected Result:** HTTP 422, response body contains `detail` with field-level errors

**Pass Criteria:** Status code 422; `detail` present in response; mentions `loan_type`

```
def test_post_application_missing_fields(client, auth_token, test_applicant):
    response = client.post("/api/v1/applications", json={
        "applicant_id": test_applicant["id"],
        # Missing: loan_type, amount_requested, tenure_months, purpose
    }, headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 422
    errors = response.json()["detail"]
    field_names = [e["loc"][-1] for e in errors]
    assert "loan_type" in field_names
```

---

TC-01-P1-API-03: GET /applications/{id} Returns Full Details

**Category:** API | **Priority:** High **Prerequisites:** Application created in test DB

**Steps:**

1. Create an application
2. GET `/api/v1/applications/{created_id}`
3. Verify response includes nested applicant and status\_history

**Expected Result:** HTTP 200, response includes `applicant` object and `status_history` list

**Pass Criteria:** Status 200; `status_history` is a list (min 1 entry for initial submission)

```
def test_get_application_by_id(client, auth_token, test_applicant):
    create_resp = client.post("/api/v1/applications", json={
        "applicant_id": test_applicant["id"],
        "loan_type": "home", "amount_requested": 2000000.0,
        "tenure_months": 120, "purpose": "Buy home"
    }, headers={"Authorization": f"Bearer {auth_token}"})
    app_id = create_resp.json()["id"]

    response = client.get(f"/api/v1/applications/{app_id}",
        headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 200
    data = response.json()
    assert data["id"] == app_id
    assert "status_history" in data
    assert len(data["status_history"]) >= 1
    assert data["status_history"][0]["new_status"] == "submitted"
```

---

#### TC-01-P1-API-04: GET /applications/{id} Not Found Returns 404

**Category:** API | **Priority:** High **Prerequisites:** Running test client

**Steps:**

1. GET `/api/v1/applications/99999` (non-existent ID)

**Expected Result:** HTTP 404

**Pass Criteria:** Status code 404; `detail` message in response

```
def test_get_application_not_found(client, auth_token):
    response = client.get("/api/v1/applications/99999",
        headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 404
    assert "detail" in response.json()
```

---

#### TC-01-P1-API-05: PATCH /status with Valid Transition Returns 200

**Category:** API | **Priority:** High **Prerequisites:** Application in "submitted" state

**Steps:**

1. Create application (status=submitted)
2. PATCH `/api/v1/applications/{id}/status` with `new_status="under_review"`

**Expected Result:** HTTP 200, application status updated

**Pass Criteria:** Status 200; subsequent GET shows `status == "under_review"`

```
def test_patch_status_valid_transition(client, auth_token, test_applicant):
    create = client.post("/api/v1/applications", json={
        "applicant_id": test_applicant["id"], "loan_type": "auto",
        "amount_requested": 500000.0, "tenure_months": 48, "purpose": "Car"
    }, headers={"Authorization": f"Bearer {auth_token}"})
    app_id = create.json()["id"]

    response = client.patch(f"/api/v1/applications/{app_id}/status", json={
        "new_status": "under_review", "remarks": "Documents look good, starting
review"
    }, headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 200

    # Verify status changed
    get_resp = client.get(f"/api/v1/applications/{app_id}",
        headers={"Authorization": f"Bearer {auth_token}"})
    assert get_resp.json()["status"] == "under_review"
```

---

TC-01-P1-API-06: PATCH /status Without JWT Returns 401

**Category:** API | **Priority:** High **Prerequisites:** Application exists

**Steps:**

1. PATCH `/api/v1/applications/1/status` WITHOUT Authorization header

**Expected Result:** HTTP 401

**Pass Criteria:** Status code 401

```
def test_patch_status_unauthorized(client):
    # No auth header
    response = client.patch("/api/v1/applications/1/status", json={
        "new_status": "under_review", "remarks": "test"
    })
    assert response.status_code == 401
```

---

TC-01-P1-API-07: GET /applications with Filters Returns Filtered Results

**Category:** API | **Priority:** High **Prerequisites:** Multiple applications with different statuses created

**Steps:**

1. Create 3 applications: 2 "submitted", 1 "under\_review" (update one)
2. GET `/api/v1/applications?status=submitted`
3. Verify only "submitted" applications are returned

**Expected Result:** Response contains only applications matching the filter

**Pass Criteria:** All returned applications have `status == "submitted"`; count = 2

```
def test_list_applications_with_filters(client, auth_token, test_applicant):
    # Create 2 submitted applications
    for i in range(2):
        client.post("/api/v1/applications", json={
            "applicant_id": test_applicant["id"], "loan_type": "personal",
            "amount_requested": 50000.0, "tenure_months": 12,
            "purpose": f"Purpose {i}"
        }, headers={"Authorization": f"Bearer {auth_token}"})

    response = client.get("/api/v1/applications?status=submitted",
                          headers={"Authorization": f"Bearer {auth_token}"})

    assert response.status_code == 200
    data = response.json()
    items = data.get("items", data if isinstance(data, list) else [])
    assert all(item["status"] == "submitted" for item in items)
    assert len(items) >= 2
```

---

TC-01-P1-API-08: GET /dashboard/summary Returns Correct Counts

**Category:** API | **Priority:** Medium **Prerequisites:** At least 1 application exists

**Steps:**

1. Create 1 application
2. GET `/api/v1/dashboard/summary`
3. Verify response structure

**Expected Result:** HTTP 200, response contains `total_applications`, `by_status`, `by_loan_type`

**Pass Criteria:** Status 200; `total_applications >= 1`; `by_status` and `by_loan_type` are dicts

```
def test_get_dashboard_summary(client, auth_token, test_applicant):
    client.post("/api/v1/applications", json={
        "applicant_id": test_applicant["id"], "loan_type": "personal",
        "amount_requested": 100000.0, "tenure_months": 12, "purpose": "Test"
    }, headers={"Authorization": f"Bearer {auth_token}"})
```

```
response = client.get("/api/v1/dashboard/summary",
                      headers={"Authorization": f"Bearer {auth_token}"})

assert response.status_code == 200
data = response.json()
assert "total_applications" in data
assert data["total_applications"] >= 1
assert "by_status" in data
assert "by_loan_type" in data
assert isinstance(data["by_status"], dict)
```

---

## DATABASE TESTS (4 cases)

### TC-01-P1-DB-01: Application Record Persisted with Correct Fields

**Category:** Database | **Priority:** High **Prerequisites:** SQLAlchemy session available

#### Steps:

1. Create application via service layer
2. Query database directly for the application by ID
3. Compare database record with input data

**Expected Result:** All fields stored correctly; `status = "submitted"`; timestamps populated

```
def test_application_created_in_db(db_session, test_user):
    from app.models.applicant import Applicant
    from app.models.application import LoanApplication, ApplicationStatus

    # Create applicant directly
    applicant = Applicant(name="DB Test", email="db@test.com", phone="1234567890",
                          annual_income=500000, employment_status="salaried")
    db_session.add(applicant)
    db_session.commit()

    # Create application
    app = LoanApplication(applicant_id=applicant.id, loan_type="personal",
                          amount_requested=100000, tenure_months=12,
                          purpose="Test")
    db_session.add(app)
    db_session.commit()

    # Query and verify
    db_app = db_session.query(LoanApplication).filter(LoanApplication.id ==
app.id).first()
    assert db_app is not None
    assert db_app.status == ApplicationStatus.submitted
    assert db_app.amount_requested == 100000
    assert db_app.submitted_at is not None
```

---

## TC-01-P1-DB-02: Status History Recorded on Status Change

**Category:** Database | **Priority:** High **Prerequisites:** Application exists, status update via service

### Steps:

1. Create application (creates initial STATUS\_HISTORY record)
2. Update status via service to "under\_review"
3. Query STATUS\_HISTORY for that application

**Expected Result:** 2 records in STATUS\_HISTORY: one for "submitted", one for "under\_review"

```
def test_status_history_recorded(db_session):
    from app.models.application import LoanApplication, ApplicationStatus
    from app.models.status_history import StatusHistory

    # Create application
    app = LoanApplication(applicant_id=1, loan_type="personal",
                          amount_requested=50000, tenure_months=12,
    purpose="Test")
    db_session.add(app)
    # Add initial history
    history = StatusHistory(application_id=app.id, old_status=None,
                            new_status=ApplicationStatus.submitted,
                            changed_by="system", remarks="Submitted")
    db_session.add(history)
    db_session.commit()

    # Update status
    app.status = ApplicationStatus.under_review
    history2 = StatusHistory(application_id=app.id,
                             old_status=ApplicationStatus.submitted,
                             new_status=ApplicationStatus.under_review,
                             changed_by="officer@test.com",
                             remarks="Starting review")
    db_session.add(history2)
    db_session.commit()

    records = db_session.query(StatusHistory).filter(
        StatusHistory.application_id == app.id
    ).all()
    assert len(records) == 2
    assert records[1].old_status == ApplicationStatus.submitted
    assert records[1].new_status == ApplicationStatus.under_review
    assert records[1].changed_by == "officer@test.com"
```

---

## TC-01-P1-DB-03: Document Upload Persisted and Linked to Application

**Category:** Database | **Priority:** Medium **Prerequisites:** Application exists

**Steps:**

1. Create a Document record linked to an application
2. Query documents for that application

**Expected Result:** Document found, linked to correct application, `verified=False` by default

```
def test_document_upload_persisted(db_session):
    from app.models.application import LoanApplication
    from app.models.document import Document, DocumentType

    app = LoanApplication(applicant_id=1, loan_type="home",
                          amount_requested=2000000, tenure_months=120,
                          purpose="House")
    db_session.add(app)
    db_session.commit()

    doc = Document(application_id=app.id, doc_type=DocumentType.id_proof,
                   file_name="aadhaar_card.pdf")
    db_session.add(doc)
    db_session.commit()

    saved_doc = db_session.query(Document).filter(
        Document.application_id == app.id
    ).first()
    assert saved_doc is not None
    assert saved_doc.doc_type == DocumentType.id_proof
    assert saved_doc.verified == False
    assert saved_doc.file_name == "aadhaar_card.pdf"
```

---

## TC-01-P1-DB-04: Cascade Delete Removes Documents

**Category:** Database | **Priority:** Medium **Prerequisites:** Application with documents exists

**Steps:**

1. Create application with 2 documents
2. Delete the application
3. Query documents for that application\_id

**Expected Result:** Documents are also deleted (cascade delete)

```
def test_cascade_delete_documents(db_session):
    from app.models.application import LoanApplication
    from app.models.document import Document, DocumentType

    app = LoanApplication(applicant_id=1, loan_type="personal",
                          amount_requested=50000, tenure_months=12,
                          purpose="Test")
```

```
db_session.add(app)
db_session.commit()
app_id = app.id

for doc_type in [DocumentType.id_proof, DocumentType.income_proof]:
    doc = Document(application_id=app_id, doc_type=doc_type,
file_name="test.pdf")
    db_session.add(doc)
    db_session.commit()

# Verify 2 docs exist
assert db_session.query(Document).filter(Document.application_id ==
app_id).count() == 2

# Delete application
db_session.delete(app)
db_session.commit()

# Verify documents also deleted
assert db_session.query(Document).filter(Document.application_id ==
app_id).count() == 0
```

---

## Running the Tests

```
# Install test dependencies
pip install pytest pytest-asyncio httpx pytest-cov

# Run all Phase 1 tests
pytest tests/ -v -k "P1"

# Run with coverage
pytest tests/ -v --cov=app --cov-report=html

# Generate JUnit XML for submission
pytest tests/ --junitxml=results/phase1-results.xml

# Run only unit tests
pytest tests/test_unit/ -v

# Run only API tests
pytest tests/test_api/ -v
```

## Expected Passing Score for Phase 1 Completion

| Category | Tests | Min to Pass |
|----------|-------|-------------|
| Unit     | 8     | 6           |
| API      | 8     | 6           |

| Category | Tests | Min to Pass |
|----------|-------|-------------|
| Database | 4     | 2           |
| Total    | 20    | 14 (70%)    |